

CineRate Database Documentation

Database Systems — Database Documentation Project

Student Name: Shimaa Nashat Instructor: Abdelrahman Mahmoud

1. Project Overview

CineRate is a relational database supporting a movie rating and review platform. It manages a movie catalogue (Movies, Directors, Actors, Genres), user accounts (Users), and user activity (Ratings, Reviews, Watchlist). Movies is the central table: Directors link to it directly (1:N); Genres and Actors link to it through junction tables MovieGenres and MovieActors (M:N); and Users interact with it through three separate 1:N activity tables.

2. Database Overview

Item	Details
Database Name	CineRate
Tables	10 (Directors, Movies, Users, Actors, Genres, Ratings, Reviews, Watchlist, MovieGenres, MovieActors)
Relationships	9 total — 7 one-to-many; 2 many-to-many via junction tables
Keys	10 primary keys (8 IDENTITY surrogate keys; 2 composite keys)
Constraints	3 CHECK ; 5 UNIQUE ; 4 DEFAULT

Table 1 — Database summary.

3. Entity-Relationship Diagram

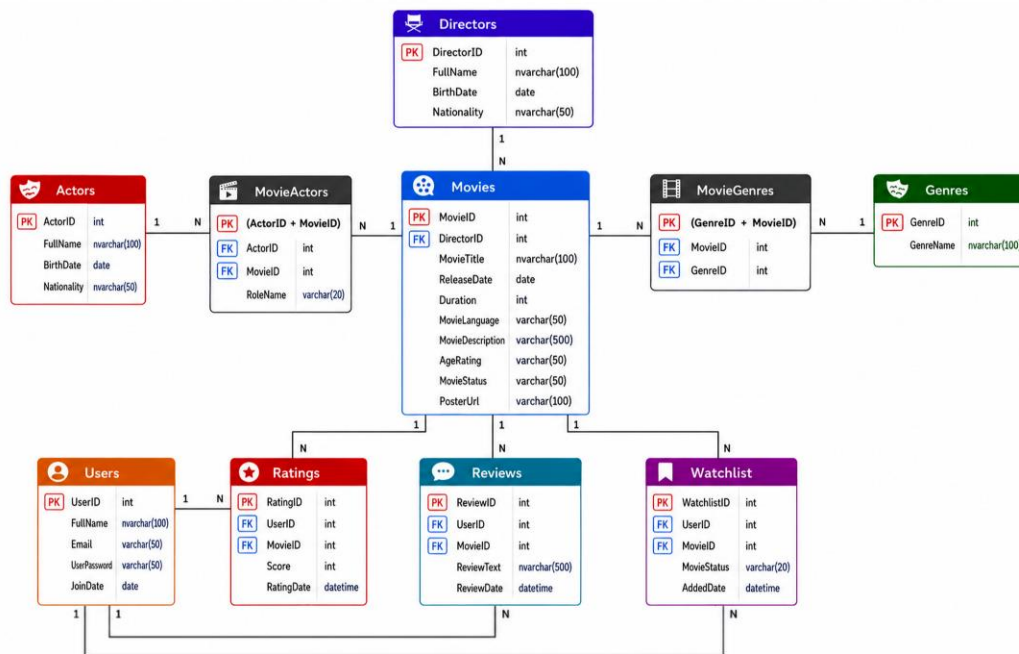


Figure 1 — CineRate ERD.

4. Tables & Columns

Table	Columns (Name : Type — Key/Notes)
Directors	DirectorID:INT PK IDENTITY • FullName:NVARCHAR(100) NN • BirthDate:DATE • Nationality:NVARCHAR(50) NN
Movies	MovieID:INT PK IDENTITY • DirectorID:INT FK • MovieTitle:NVARCHAR(100) NN • ReleaseDate:DATE NN • Duration:INT • MovieLanguage:VARCHAR(50) NN • MovieDescription:VARCHAR(500) NN • AgeRating:VARCHAR(50) NN • MovieStatus:VARCHAR(50) CHECK('Available','Archived') • PosterUrl:VARCHAR(100) NN
Users	UserID:INT PK IDENTITY • FullName:NVARCHAR(100) NN • Email:VARCHAR(50) NN UNIQUE • UserPassword:VARCHAR(50) NN • JoinDate:DATE DEFAULT GETDATE()
Actors	ActorID:INT PK IDENTITY • FullName:NVARCHAR(100) NN • BirthDate:DATE • Nationality:NVARCHAR(50) NN
Genres	GenreID:INT PK IDENTITY • GenreName:NVARCHAR(100) NN UNIQUE
Ratings	RatingID:INT PK IDENTITY • UserID:INT FK • MovieID:INT FK • Score:INT NN CHECK(1–10) • RatingDate:DATETIME DEFAULT GETDATE() • UNIQUE(UserID,MovieID)
Reviews	ReviewID:INT PK IDENTITY • UserID:INT FK • MovieID:INT FK • ReviewText:NVARCHAR(500) NN • ReviewDate:DATETIME DEFAULT GETDATE() • UNIQUE(UserID,MovieID)
Watchlist	WatchlistID:INT PK IDENTITY • UserID:INT FK • MovieID:INT FK • MovieStatus:VARCHAR(20) NN CHECK('Planned','Watching','Watched') • AddedDate:DATETIME DEFAULT GETDATE() • UNIQUE(UserID,MovieID)
MovieGenres	GenreID:INT PK,FK • MovieID:INT PK,FK — composite PK PK_MovieGenres (M:N junction)
MovieActors	ActorID:INT PK,FK • MovieID:INT PK,FK • RoleName:VARCHAR(20) NN — composite PK PK_MovieActors (M:N junction)

Table 2 — All tables and columns (NN = NOT NULL).

5. Keys & Relationships

Parent → Child	Type	FK / Junction	Notes
Directors → Movies	1:N	Movies.DirectorID	One director directs many movies (nullable FK).
Users → Ratings	1:N	Ratings.UserID	One user submits many ratings.
Movies → Ratings	1:N	Ratings.MovieID	One movie receives many ratings.
Users → Reviews	1:N	Reviews.UserID	One user writes many reviews.
Movies → Reviews	1:N	Reviews.MovieID	One movie receives many reviews.
Users → Watchlist	1:N	Watchlist.UserID	One user has many watchlist entries.
Movies → Watchlist	1:N	Watchlist.MovieID	One movie appears on many watchlists.
Movies ↔ Genres	M:N	via MovieGenres	Resolved through junction table (GenreID+MovieID composite PK).
Movies ↔ Actors	M:N	via MovieActors	Resolved through junction table (ActorID+MovieID composite PK), plus RoleName.

Table 3 — All 9 relationships. All 10 tables have explicit primary keys; 11 FKs link them (no ON DELETE/UPDATE actions specified, default NO ACTION).

6. Constraints Summary

- CHECK: Movies.MovieStatus ∈ {Available, Archived} · Ratings.Score BETWEEN 1–10 · Watchlist.MovieStatus ∈ {Planned, Watching, Watched}
- UNIQUE: Users.Email · Genres.GenreName · named composite UNIQUE(UserID, MovieID) on Ratings, Reviews, and Watchlist

- DEFAULT: GETDATE() on Users.JoinDate, Ratings.RatingDate, Reviews.ReviewDate, Watchlist.AddedDate
- IDENTITY(1,1): auto-increment on all 8 single-column primary keys

7. ERD vs SQL Consistency

All 10 tables, all columns, all 10 primary keys, and all 11 foreign keys/relationships match exactly between the ERD and the SQL script — no discrepancies found. The only items present in the SQL but not depicted in the ERD (as expected for ERD notation) are: CHECK constraints, UNIQUE constraints, DEFAULT values, NOT NULL/nullability, and IDENTITY specifications.

8. Sample SQL Queries — With Verified Output

All 10 queries below were executed against the actual loaded dataset (60 directors, 220 actors, 20 genres, 700 movies, 150 users, 1,213 movie-genre links, 3,140 movie-actor links, 2,200 ratings, 900 reviews, 1,000 watchlist entries) to confirm they run correctly and return the expected shape of result. Result tables are capped at 5 sample rows (or fewer) for readability.

Q1 — Available movies, newest first

```
SELECT * FROM Movies
WHERE MovieStatus = 'Available'
ORDER BY ReleaseDate DESC;
```

MovieID	MovieTitle	ReleaseDate	MovieStatus
411	The Bridge Chronicles	2025-10-09	Available
645	Crimson Nights	2025-08-01	Available
597	Return to Silver	2025-07-20	Available
458	The Tower	2025-05-08	Available
241	Beyond the Wraith	2025-04-23	Available

Q2 — Distinct movie languages

```
SELECT DISTINCT MovieLanguage FROM Movies;
```

MovieLanguage
English
Korean
Swedish
Spanish
Turkish
French
Italian
Japanese
Portuguese
German
Mandarin
Hindi
Russian

MovieLanguage
Arabic

Q3 — Directors filtered by nationality (IN)

```
SELECT * FROM Directors
WHERE Nationality IN ('Egyptian', 'French');
```

DirectorID	FullName	Nationality
2	Barbara Clark	French
4	Magnus Miller	Egyptian
7	Karen Yang	French
9	Daniel Abubakar	French
39	Kwame Bello	French
50	Kenneth Adeyemi	Egyptian

Q4 — Overall rating statistics

```
SELECT COUNT(*) AS TotalRatings,
       MIN(Score) AS LowestScore,
       MAX(Score) AS HighestScore,
       AVG(Score) AS AvgScore
FROM Ratings;
```

TotalRatings	LowestScore	HighestScore	AvgScore
2200	1	10	5.35

Q5 — Ratings between 7 and 10 (BETWEEN)

```
SELECT RatingID, MovieID, UserID, Score
FROM Ratings
WHERE Score BETWEEN 7 AND 10
ORDER BY Score DESC;
```

RatingID	MovieID	UserID	Score
9	265	95	10
20	485	109	10
32	538	77	10
55	25	55	10
80	4	77	10

Q6 — Titles starting with 'The' (LIKE)

```
SELECT MovieID, MovieTitle
FROM Movies
WHERE MovieTitle LIKE 'The%';
```

MovieID	MovieTitle
7	The Orchard Chronicles

MovieID	MovieTitle
8	The Whisper Diaries
9	The Fury
11	The Arrow Diaries
16	The Voyage

Q7 — Ratings count, min & average per movie (JOIN + GROUP BY)

```
SELECT Movies.MovieID, Movies.MovieTitle,
       COUNT(Ratings.RatingID) AS NumberOfRatings,
       MIN(Ratings.Score) AS LowestScore,
       AVG(Ratings.Score) AS AvgScore
FROM Movies
INNER JOIN Ratings ON Movies.MovieID = Ratings.MovieID
GROUP BY Movies.MovieID, Movies.MovieTitle;
```

MovieID	MovieTitle	NumberOfRatings	LowestScore	AvgScore
1	A Broken Crown	6	2	3.83
2	Children of Legacy	3	1	3.33
3	Warm Nights	3	2	3.33
4	Beyond the Autumn	4	3	6.75
5	Children of Autumn	4	3	6.0

Q8 — Movies longer than the average duration (subquery)

```
SELECT * FROM Movies
WHERE Duration > (
    SELECT AVG(Duration)
    FROM Movies
);
```

MovieID	MovieTitle	Duration
8	The Whisper Diaries	151
9	The Fury	133
10	Song of the Tower	177
12	Lost Nights	137
13	Compass's Edge	134

Q9 — Top 3 highest-rated movies (JOIN + GROUP BY + ORDER BY + TOP)

```
SELECT TOP 3 Movies.MovieTitle, AVG(Ratings.Score) AS AverageRating
FROM Movies
INNER JOIN Ratings ON Movies.MovieID = Ratings.MovieID
GROUP BY Movies.MovieID, Movies.MovieTitle
ORDER BY AverageRating DESC;
```

MovieTitle	AverageRating
Tide's Edge	10.0
Return to River	10.0

MovieTitle	AverageRating
Beyond the Crown	10.0

Q10 — Movies, their actors, and roles (double JOIN)

```
SELECT Movies.MovieTitle,
       Actors.FullName AS ActorName,
       MovieActors.RoleName
FROM Movies
INNER JOIN MovieActors ON Movies.MovieID = MovieActors.MovieID
INNER JOIN Actors ON MovieActors.ActorID = Actors.ActorID;
```

MovieTitle	ActorName	RoleName
A Broken Crown	Anders Girard	Lead Actress
A Broken Crown	Layla Esposito	Supporting
A Broken Crown	Rohan Silva	Supporting Actor
A Broken Crown	Ahmed Yang	Guest Star
Children of Legacy	Karim Rao	Lead

9. Normalization Analysis

The CineRate schema was checked against the first three normal forms. All 10 tables satisfy 3NF:

1NF — Atomic values, no repeating groups

- Every column holds a single atomic value (e.g. Nationality is one value per row, not a list).
- Repeating relationships — a movie having many genres, or many actors — are resolved into separate junction rows (MovieGenres, MovieActors) rather than repeating columns or comma-separated values.
- Every table has a defined primary key, so no duplicate rows can exist.

2NF — No partial dependency on a composite key

- Only two tables have composite primary keys: MovieGenres (GenreID + MovieID) and MovieActors (ActorID + MovieID).
- MovieGenres has no non-key attributes, so partial dependency cannot occur.
- MovieActors' one non-key attribute, RoleName, describes the pairing of that specific actor with that specific movie — it depends on the whole composite key, not on ActorID or MovieID alone — so 2NF holds.
- All other tables use a single-column surrogate key, so partial dependency does not apply to them.

3NF — No transitive dependency on non-key attributes

- In Movies, attributes such as MovieTitle, ReleaseDate, Duration, and AgeRating each depend only on MovieID, not on one another. DirectorID is a foreign key reference, not a derived value.
- In Ratings, Reviews, and Watchlist, all non-key attributes (Score, ReviewText, MovieStatus, dates) depend directly on the record's own surrogate key, not on UserID or MovieID individually.
- Director and Actor biographical data (BirthDate, Nationality) live only in Directors/Actors and are referenced by key from Movies/MovieActors — they are not duplicated into the movie tables, avoiding transitive update anomalies.

Conclusion: the schema is in 3NF. The two junction tables are the correct, minimal way to resolve the two many-to-many relationships (Movies–Genres and Movies–Actors) without redundancy.

10. Referential Integrity Notes

All 11 foreign keys in the script are declared without an explicit ON DELETE or ON UPDATE clause, so SQL Server applies the default behavior, NO ACTION, in every case. Practical implications:

- Deleting a Movie, User, Director, Actor, or Genre that still has dependent rows (Ratings, Reviews, Watchlist, MovieGenres, MovieActors, or Movies itself for Directors) will be rejected by the engine rather than cascading.
- Application code (or a future migration) needs to delete or reassign child rows first — e.g. remove a movie's ratings, reviews, watchlist entries, genre links, and actor links — before the parent Movie row can be removed.
- This is a safe default for a rating/review platform: it prevents a single accidental delete from silently wiping out a user's whole history.